



SECTION 4 – Parallel & Distributed Computing Message Passing Interface (MPI)

Ir. Randi Rizal, Ph.D.



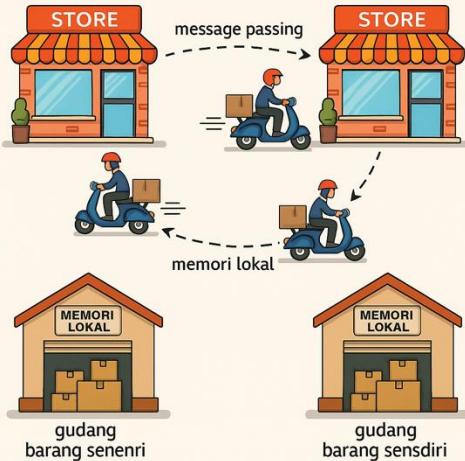
Apa itu MPI?

- ✓ **MPI (Message Passing Interface)** adalah standar untuk melakukan komunikasi antar proses pada sistem **paralel dan terdistribusi**.
- ✓ Digunakan di **cluster komputer, superkomputer, atau multi-node environment**.
- ✓ Tidak seperti thread yang berbagi memori, MPI menggunakan model **distributed memory**.
- ✓ Setiap proses punya **memori sendiri**, sehingga tidak bisa langsung mengakses data proses lain.
- ✓ Pertukaran data dilakukan dengan mengirim dan menerima pesan (**send/receive**).



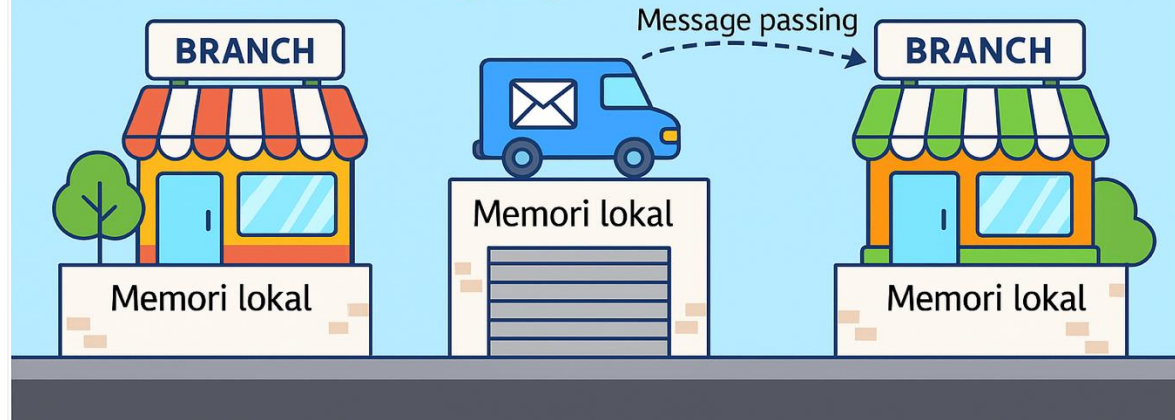
1 Contoh Analogi

- Bayangkan ada banyak toko cabang (proses).
- Setiap cabang punya gudang barang sendiri (memori lokal).
- Kalau butuh barang dari cabang lain, mereka harus mengirim permintaan via kurir (*message passing*), bukan ambil langsung.



2 Contoh Analogi

- Bayangkan ada banyak toko cabang (proses).
- Setiap cabang punya **gudang barang** sendiri (memoi lokal).
- Kalau butuh barang dari cabang lain, mereka harus mengirim **permintaan via kurir** (*message passing*), bukan ambil langsung.





- ✓ **Program MPI** biasanya berjalan dengan banyak proses (disebut **rank**) yang diluncurkan **secara bersamaan**.
- ✓ Proses-proses ini berkomunikasi melalui jaringan (interconnect) dengan operasi point-to-point atau collective.
- ✓ Semua proses berjalan **independen**, tetapi bisa berkoordinasi **lewat pesan**.

Keunggulan:

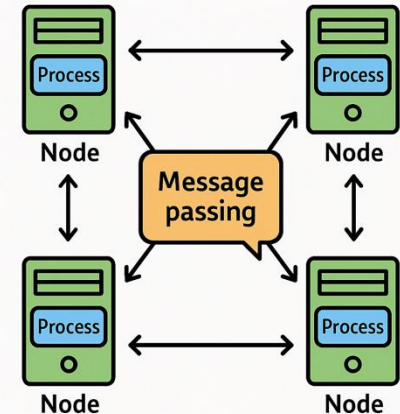
- ✓ **Portabilitas**: Bisa dijalankan di berbagai platform (cluster, supercomputer, bahkan laptop multi-core).
- ✓ **Skalabilitas**: Dapat berjalan pada ratusan hingga ribuan node.
- ✓ **Fleksibilitas**: Mendukung komunikasi point-to-point maupun collective.





- **MPI** = standar komunikasi antar proses di sistem parallel/terdistribusi.
- Beda dengan `thread` atau `multiprocessing`, di **MPI setiap proses punya memori sendiri.**
- Proses berkomunikasi lewat **message passing** (kirim & terima pesan).
- Digunakan pada **cluster komputer** dan **supercomputer**, tapi bisa dijalankan di laptop untuk belajar.

Konsep Dasar MPI



MPI memungkinkan setiap node pada sistem komputasi saling mengirim/menerima data (*messages*).



Konsep Dasar MPI (Message Passing Interface) -> Send & Receive



Program 1: Halo dari Setiap Proses

```
# mpi_hello.py
from mpi4py import MPI

comm = MPI.COMM_WORLD # dunia MPI
rank = comm.Get_rank() # ID proses (0,1,2,...)
size = comm.Get_size() # jumlah total proses

print(f"Halo dari proses {rank} dari total {size} proses")
```

***Kode Program**

```
mpiexec -n 4 python mpi_hello.py
```

```
Halo dari proses 0 dari total 4 proses
Halo dari proses 1 dari total 4 proses
Halo dari proses 2 dari total 4 proses
Halo dari proses 3 dari total 4 proses
```

Output

Artinya ada 4 proses berbeda yang berjalan bersamaan.

Program 2: Send & Receive

```
# mpi_send_recv.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    data = "Pesan dari proses 0"
    comm.send(data, dest=1) # kirim ke proses 1
    print("Proses 0 mengirim pesan")
elif rank == 1:
    data = comm.recv(source=0) # terima dari proses 0
    print("Proses 1 menerima:", data)
```

***Kode Program**

```
mpiexec -n 2 python mpi_send_recv.py
```

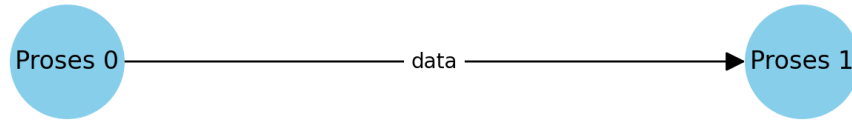
```
Proses 0 mengirim pesan
Proses 1 menerima: Pesan dari proses 0
```

Output

Terlihat proses 0 mengirim pesan ke proses 1.



Ilustrasi MPI Send/Receive



Ini adalah ilustrasi sederhana **MPI Send/Receive**:

- ✓ **Proses 0** mengirim data ke **Proses 1**.
- ✓ Jalurnya ditunjukkan dengan panah → data bergerak dari satu proses ke proses lain.



Program 3: Broadcast

```
# mpi_broadcast.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    data = "Halo semua, dari Proses 0"
else:
    data = None

data = comm.bcast(data, root=0) # broadcast dari root=0
print(f"Proses {rank} menerima: {data}")
```

```
mpiexec -n 4 python mpi_broadcast.py
```

```
Proses 0 menerima: Halo semua, dari Proses 0
Proses 1 menerima: Halo semua, dari Proses 0
Proses 2 menerima: Halo semua, dari Proses 0
Proses 3 menerima: Halo semua, dari Proses 0
```

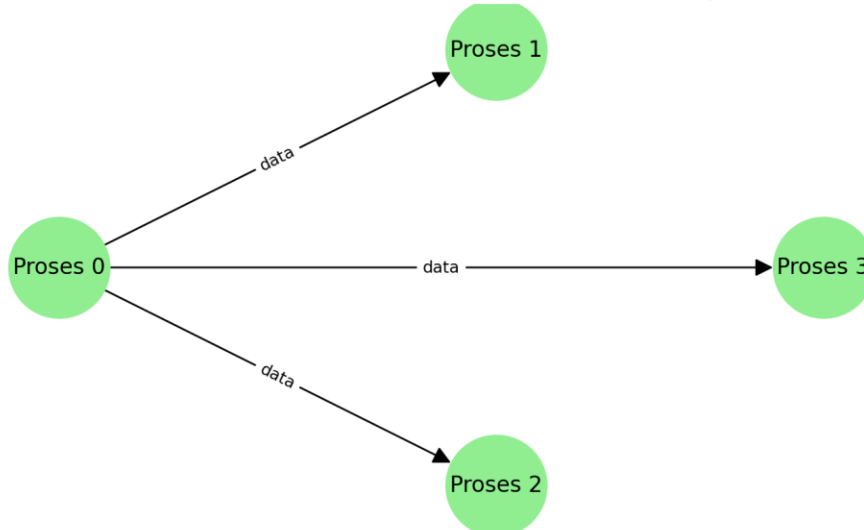
Semua proses menerima pesan dari proses 0.

Analogi Mudah

- ✓ Bayangkan ada kelas dengan banyak mahasiswa.
- ✓ MPI seperti WhatsApp Group:
 - Mahasiswa (proses) bisa chat langsung ke 1 orang (send/recv).
 - Bisa juga broadcast ke semua orang (bcast).



Ilustrasi MPI Broadcast (Proses 0 ke semua proses)



Berikut ilustrasi **MPI Broadcast**:

- ✓ **Proses 0** mengirim **data yang sama** ke semua proses lain (Proses 1, 2, dan 3).
- ✓ Semua proses menerima pesan dari **root (Proses 0)**.

Dengan broadcast, komunikasi jadi lebih efisien dibandingkan mengirim satu per satu (send/receive).



Konsep Dasar MPI (Message Passing Interface) -> Scatter & Gather



```
# mpi_scatter.py
from mpi4py import MPI
```

Code: Scatter

```
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    data = [x for x in range(size)] # data = [0,1,2,3,...]
else:
    data = None

# Scatter: Proses 0 membagi data ke semua proses
recv_data = comm.scatter(data, root=0)
print(f"Proses {rank} menerima data: {recv_data}")
```

```
mpiexec -n 4 python mpi_scatter.py
```

```
Proses 0 menerima data: 0
Proses 1 menerima data: 1
Proses 2 menerima data: 2
Proses 3 menerima data: 3
```

Output

Data [0,1,2,3] dibagi ke 4 proses.

```
# mpi_gather.py
from mpi4py import MPI
```

Code: Gather

```
comm = MPI.COMM_WORLD
rank = comm.Get_rank()

# Setiap proses punya data masing-masing
send_data = rank * 10

# Gather: Semua proses kirim data ke root=0
gathered = comm.gather(send_data, root=0)

if rank == 0:
    print("Proses 0 mengumpulkan data:", gathered)
```

```
mpiexec -n 4 python mpi_gather.py
```

```
Proses 0 mengumpulkan data: [0, 10, 20, 30]
```

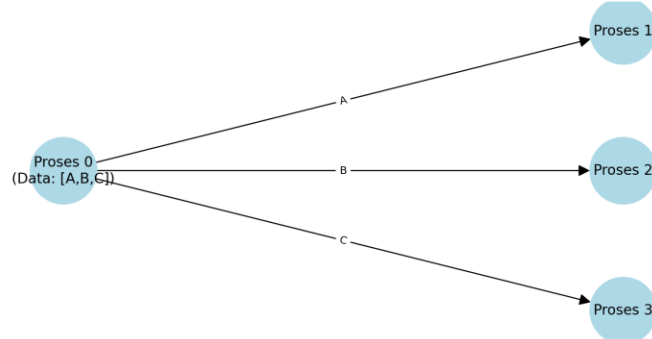
Semua proses mengirim data ke Proses 0 untuk dikumpulkan.



Konsep Dasar MPI (Message Passing Interface) -> Scatter & Gather



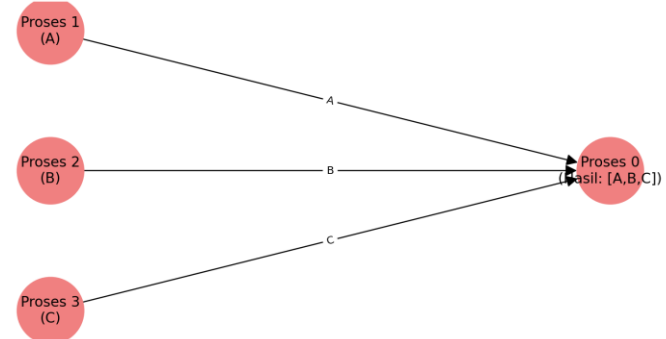
Ilustrasi MPI Scatter (Distribusi Data dari Proses 0)



MPI Scatter

- ✓ **Proses 0** punya data [A, B, C].
- ✓ Data didistribusikan ke proses lain:
 - Proses 1 → A
 - Proses 2 → B
 - Proses 3 → C

Ilustrasi MPI Gather (Pengumpulan Data ke Proses 0)



MPI Gather

- ✓ Setiap proses punya data masing-masing (A, B, C).
- ✓ Semua proses mengirim data kembali ke **Proses 0**.
- ✓ Proses 0 mengumpulkan hasil menjadi [A, B, C].

Scatter & Gather sering dipakai untuk **pembagian kerja** dan **penggabungan hasil** pada sistem paralel.



Konsep Dasar MPI (Message Passing Interface) -> Scatter & Gather



Code: Scatter + Gather (gabungan)

```
# mpi_scatter_gather.py
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    data = [i for i in range(size)] # [0,1,2,3,...]
else:
    data = None

# Scatter
recv_data = comm.scatter(data, root=0)
print(f"Proses {rank} menerima data: {recv_data}")

# Masing-masing proses mengolah datanya
result = recv_data ** 2

# Gather
results = comm.gather(result, root=0)

if rank == 0:
    print("Hasil pengumpulan di Proses 0:", results)
```

```
mpiexec -n 4 python mpi_scatter_gather.py
```

```
Proses 0 menerima data: 0
Proses 1 menerima data: 1
Proses 2 menerima data: 2
Proses 3 menerima data: 3
Hasil pengumpulan di Proses 0: [0, 1, 4, 9]
```

Data dibagi dengan **Scatter**, dihitung di tiap proses, lalu dikumpulkan kembali dengan **Gather**.



Soal 1. Scatter Sederhana:

Tulislah program MPI menggunakan `scatter` yang:

- ✓ Proses 0 memiliki list `[10, 20, 30, 40]`.
- ✓ List tersebut dibagi ke 4 proses menggunakan `scatter`.
- ✓ Setiap proses mencetak data yang diterimanya.

Pertanyaan:

1. Apa output yang dihasilkan tiap proses?
2. Mengapa urutannya bisa berbeda saat dicetak?



Soal 2. Gather Sederhana:

Tulislah program MPI menggunakan `gather` yang:

- ✓ Setiap proses mengirim data berupa $\text{rank} * 5$.
- ✓ Data dikumpulkan di proses 0.
- ✓ Proses 0 mencetak hasil pengumpulan.

Pertanyaan:

1. Apa isi list hasil di proses 0 jika ada 4 proses?
2. Bagaimana hasilnya jika jumlah proses = 6?



Soal 3. Scatter + Gather Kombinasi:

Buatlah program yang:

- Proses 0 memiliki list [1,2,3,4].
- Data dibagi ke semua proses dengan scatter.
- Setiap proses menghitung **kuadrat** dari datanya.
- Hasil dikumpulkan kembali ke proses 0 dengan gather.

Pertanyaan:

- Apa hasil akhir yang dikumpulkan di proses 0 ?
- Bagaimana jika list awal diganti [2,4,6,8] ?



Soal 4. Hitung Jumlah dengan Scatter + Gather (Bonus)

- ✓ Proses 0 punya list [5,10,15,20].
- ✓ Data disebar dengan scatter.
- ✓ Setiap proses menghitung $2 \times \text{datanya}$.
- ✓ Hasil dikumpulkan dan dijumlahkan di proses 0.

Jawaban Singkat: Hasil akhir di proses 0 = 100 (karena $(2 \times 5) + (2 \times 10) + (2 \times 15) + (2 \times 20)$).



- ✓ Memahami konsep **collective communication** dalam MPI.
- ✓ Mempelajari perbedaan **Reduce** dan **Allreduce**.
- ✓ Mampu mengimplementasikan operasi global (sum, max, average) dg mpi4py.
- ✓ Melihat manfaat Reduce (hasil di root) vs Allreduce (hasil di semua proses).
- ✓ **MPI_Reduce**: menggabungkan nilai dari semua proses menggunakan operasi tertentu (SUM, MAX, MIN, PROD, dll.) dan mengirim hasil ke **satu proses root**.
- ✓ **MPI_Allreduce**: sama seperti Reduce, tapi hasil **dibagikan ke semua proses**.
- ✓ **Kapan dipakai?**
 - Reduce → hanya root yang butuh hasil (misal untuk logging).
 - Allreduce → semua proses butuh hasil (misal untuk normalisasi data, error global).



Konsep Dasar MPI (Message Passing Interface) -> Reduce vs Allreduce



```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Set nilai lokal di setiap proses (misalnya sama dengan rank)
local_value = rank + 1
print(f"Rank {rank} punya nilai lokal = {local_value}")

# ----- Reduce -----
# Root process (rank 0) akan menerima hasil penjumlahan
sum_reduce = comm.reduce(local_value, op=MPI.SUM, root=0)

if rank == 0:
    print(f"[Reduce] Jumlah total dari semua proses = {sum_reduce}")

# ----- Allreduce -----
# Semua proses menerima hasil penjumlahan
sum_allreduce = comm.allreduce(local_value, op=MPI.SUM)
print(f"[Allreduce] Rank {rank} melihat jumlah total = {sum_allreduce}")
```

```
mpirun -n 4 python mpi_reduce_allreduce.py
```

```
Rank 0 punya nilai lokal = 1
Rank 1 punya nilai lokal = 2
Rank 2 punya nilai lokal = 3
Rank 3 punya nilai lokal = 4
[Reduce] Jumlah total dari semua proses = 10
[Allreduce] Rank 0 melihat jumlah total = 10
[Allreduce] Rank 1 melihat jumlah total = 10
[Allreduce] Rank 2 melihat jumlah total = 10
[Allreduce] Rank 3 melihat jumlah total = 10
```

Analisis

- ✓ **Reduce** → hanya rank 0 yang tahu hasil penjumlahan (10).
- ✓ **Allreduce** → semua rank (0,1,2,3) tahu hasil total (10).
- ✓ Jika proses lain selain root butuh nilai hasil, maka lebih efisien pakai **Allreduce** daripada Reduce + Broadcast.



1. Ubah operasi MPI.SUM menjadi MPI.MAX → proses mana yang punya nilai terbesar?
2. Hitung **rata-rata global** dengan cara:
 - ✓ Gunakan MPI.SUM untuk jumlah.
 - ✓ Bagi hasilnya dengan size.
3. Terapkan program sederhana: tiap proses menghasilkan 1000 bilangan acak, lalu gunakan Allreduce untuk menghitung **total jumlah bilangan** di seluruh proses.

THANKS!



Any questions?

You can find me at

randirizal@unsil.ac.id / +62 853 2013 2014